NYPD Motor Vehicle Collisions: Data Profiling Report

CS-404 BDA Assignment 2, Humna Tariq (460497) & Huda Noor Ahmad (458675)

Dataset: NYPD Motor Vehicle Collisions — Crashes
Source: https://data.cityofnewyork.us/Public-Safety/Motor-Vehicle-Collisions-Crashes/h9gi-nx95/about_data **Course:** CS-404 Big Data Analytics

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 100

# Load everything as strings first then it will preserve the raw
values
df_raw = pd.read_csv('./data/Motor_Vehicle_Collisions_Crashes.csv',
dtype=str, low_memory=False)
print(f'Raw load shape: {df_raw.shape}')
```

```
Raw load shape: (2255537, 29)
```

```
# Numeric columns to convert explicitly
numeric_cols = [
    'LATITUDE', 'LONGITUDE', 'COLLISION_ID',
    'NUMBER OF PERSONS INJURED', 'NUMBER OF PERSONS KILLED',
    'NUMBER OF PEDESTRIANS INJURED', 'NUMBER OF PEDESTRIANS KILLED',
    'NUMBER OF CYCLIST INJURED', 'NUMBER OF CYCLIST KILLED',
    'NUMBER OF MOTORIST INJURED', 'NUMBER OF MOTORIST KILLED',
]

df = df_raw.copy()

# convert numeric colums
# non-parseable values become NaN
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
```

```python
# Parse crash date as datetime
df['CRASH DATE'] = pd.to_datetime(df['CRASH DATE'], errors='coerce')

print(f'Working dataframe shape: {df.shape}')
print(f'Dtypes after conversion:')
print(df.dtypes)
```

```
Working dataframe shape: (2255537, 29)
Dtypes after conversion:
CRASH DATE                      datetime64[us]
CRASH TIME                                 str
BOROUGH                                    str
ZIP CODE                                   str
LATITUDE                               float64
LONGITUDE                              float64
LOCATION                                   str
ON STREET NAME                             str
CROSS STREET NAME                          str
OFF STREET NAME                            str
NUMBER OF PERSONS INJURED              float64
NUMBER OF PERSONS KILLED               float64
NUMBER OF PEDESTRIANS INJURED            int64
NUMBER OF PEDESTRIANS KILLED             int64
NUMBER OF CYCLIST INJURED                int64
NUMBER OF CYCLIST KILLED                 int64
NUMBER OF MOTORIST INJURED               int64
NUMBER OF MOTORIST KILLED                int64
CONTRIBUTING FACTOR VEHICLE 1              str
CONTRIBUTING FACTOR VEHICLE 2              str
CONTRIBUTING FACTOR VEHICLE 3              str
CONTRIBUTING FACTOR VEHICLE 4              str
CONTRIBUTING FACTOR VEHICLE 5              str
COLLISION_ID                             int64
VEHICLE TYPE CODE 1                        str
VEHICLE TYPE CODE 2                        str
VEHICLE TYPE CODE 3                        str
VEHICLE TYPE CODE 4                        str
VEHICLE TYPE CODE 5                        str
dtype: object
```

Section 1: Schema Description

All 29 columns with their inferred pandas dtype and 3 representative sample values.

```python
schema_rows = []
for col in df.columns:
    dtype = str(df[col].dtype)
```

```
    samples = df[col].dropna().head(3).tolist()
    samples_str = ' | '.join(str(s)[:30] for s in samples)
    schema_rows.append({'Column': col, 'Dtype': dtype, 'Sample Values
(first 3 non-null)': samples_str})

schema_df = pd.DataFrame(schema_rows)
schema_df.index = schema_df.index + 1

pd.set_option('display.max_colwidth', 80)
pd.set_option('display.max_rows', 35)
schema_df
```

```
                          Column           Dtype  \
1                      CRASH DATE  datetime64[us]
2                      CRASH TIME             str
3                         BOROUGH             str
4                        ZIP CODE             str
5                        LATITUDE         float64
6                       LONGITUDE         float64
7                        LOCATION             str
8                  ON STREET NAME             str
9                CROSS STREET NAME             str
10                 OFF STREET NAME             str
11      NUMBER OF PERSONS INJURED         float64
12       NUMBER OF PERSONS KILLED         float64
13  NUMBER OF PEDESTRIANS INJURED           int64
14   NUMBER OF PEDESTRIANS KILLED           int64
15      NUMBER OF CYCLIST INJURED           int64
16       NUMBER OF CYCLIST KILLED           int64
17     NUMBER OF MOTORIST INJURED           int64
18      NUMBER OF MOTORIST KILLED           int64
19  CONTRIBUTING FACTOR VEHICLE 1             str
20  CONTRIBUTING FACTOR VEHICLE 2             str
21  CONTRIBUTING FACTOR VEHICLE 3             str
22  CONTRIBUTING FACTOR VEHICLE 4             str
23  CONTRIBUTING FACTOR VEHICLE 5             str
24                   COLLISION_ID           int64
25            VEHICLE TYPE CODE 1             str
26            VEHICLE TYPE CODE 2             str
27            VEHICLE TYPE CODE 3             str
28            VEHICLE TYPE CODE 4             str
29            VEHICLE TYPE CODE 5             str

                                                       Sample Values
(first 3 non-null)
1                    2021-09-11 00:00:00 | 2022-03-26 00:00:00 | 2023-
11-01 00:00:00
2                                                                2:39 |
11:45 | 1:29
3                                                          BROOKLYN |
```

BROOKLYN | BROOKLYN
4 11230 |
11208 | 11233
5 40.62179 |
40.667202 | 40.683304
6 -73.970024 | -73.8665
| -73.917274
7 (40.62179, -73.970024) | (40.667202, -73.8665) |
(40.683304, -73...
8 WHITESTONE EXPRESSWAY | QUEENSBORO BRIDGE UPPER |
OCEAN PARKWAY
9 20 AVENUE | AVENUE K | HENRY
HUDSON RIVER
10 61 Ed Koch queensboroug | 1211 LORING AVENUE | 344
BAYCHES...
11
2.0 | 1.0 | 1.0
12
0.0 | 0.0 | 0.0
13
0 | 0 | 0
14
0 | 0 | 0
15
0 | 0 | 0
16
0 | 0 | 0
17
2 | 1 | 1
18
0 | 0 | 0
19 Aggressive Driving/Road Rage | Pavement Slippery
| Unspecified
20 Unspecified | Unspecified
| Unspecified
21 Unspecified | Unspecified
| Unspecified
22 Unspecified | Unspecified
| Unspecified
23 Unspecified | Unspecified
| Unspecified
24 4455765 |
4513547 | 4675373
25 Sedan |
Sedan | Moped
26 Sedan | Sedan |
Pick-up Truck
27 Sedan | Sedan | Station
Wagon/Sport Utility Ve

```
28          Station Wagon/Sport Utility Ve | Sedan | Station
Wagon/Sport Utility Ve
29          Station Wagon/Sport Utility Ve | Station Wagon/Sport
Utility Ve | Sedan
```

Section 2: Missing Value Analysis

Count and percentage of null values per column, sorted descending and vizualized as a horizontal bar chart.

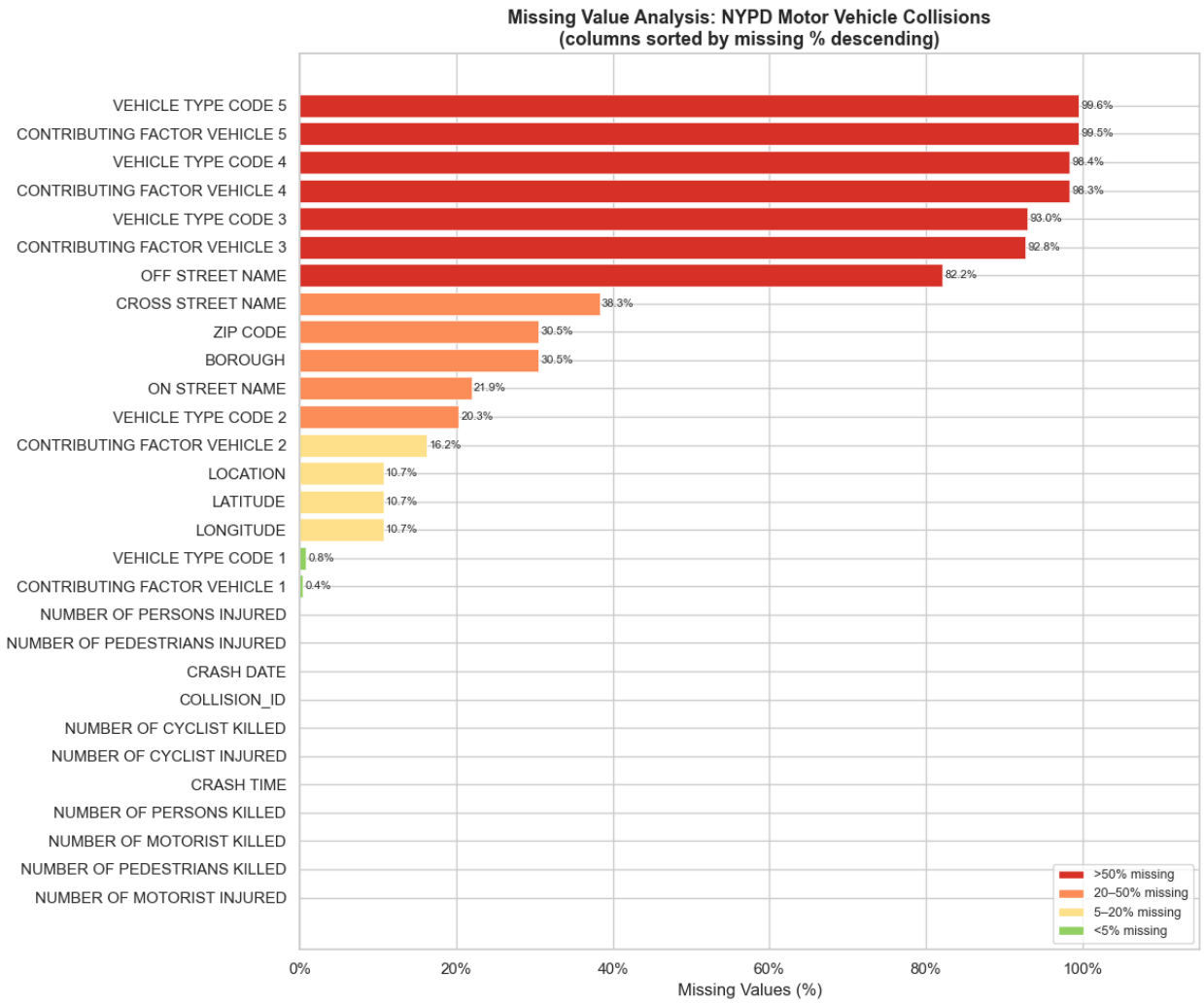
```
# Computing missing counts and percentages
total_rows = len(df)
missing_count = df.isnull().sum()
missing_pct = (missing_count / total_rows * 100).round(2)

missing_df = pd.DataFrame({
    'Missing Count': missing_count,
    'Missing %': missing_pct
}).sort_values('Missing %', ascending=False)

missing_df_nonzero = missing_df[missing_df['Missing Count'] > 0]
print(f'Columns with missing values: {len(missing_df_nonzero)} /
{len(df.columns)}')
missing_df_nonzero
```

```
Columns with missing values: 20 / 29
```

```
                                Missing Count  Missing %
VEHICLE TYPE CODE 5                   2245632    99.5600
CONTRIBUTING FACTOR VEHICLE 5         2245309    99.5500
VEHICLE TYPE CODE 4                   2219558    98.4000
CONTRIBUTING FACTOR VEHICLE 4         2218192    98.3400
VEHICLE TYPE CODE 3                   2098690    93.0500
CONTRIBUTING FACTOR VEHICLE 3         2092247    92.7600
OFF STREET NAME                       1853092    82.1600
CROSS STREET NAME                      863846    38.3000
ZIP CODE                               688348    30.5200
BOROUGH                                688065    30.5100
ON STREET NAME                         494819    21.9400
VEHICLE TYPE CODE 2                    458635    20.3300
CONTRIBUTING FACTOR VEHICLE 2          366065    16.2300
LOCATION                               240710    10.6700
LATITUDE                               240710    10.6700
LONGITUDE                              240710    10.6700
VEHICLE TYPE CODE 1                     16866     0.7500
CONTRIBUTING FACTOR VEHICLE 1            8169     0.3600
```

```
NUMBER OF PERSONS INJURED                  18     0.0000
NUMBER OF PERSONS KILLED                   31     0.0000
```

```python
# Horizontal bar chart of missing % for all columns (sorted
descending)
fig, ax = plt.subplots(figsize=(12, 10))

plot_data = missing_df['Missing %'].sort_values(ascending=True)
colors = ['#d73027' if v > 50 else '#fc8d59' if v > 20 else '#fee08b'
if v > 5 else '#91cf60'
          for v in plot_data.values]

bars = ax.barh(plot_data.index, plot_data.values, color=colors,
edgecolor='white', linewidth=0.5)

for bar, val in zip(bars, plot_data.values):
    if val > 0:
        ax.text(val + 0.3, bar.get_y() + bar.get_height() / 2,
                f'{val:.1f}%', va='center', fontsize=8)

ax.set_xlabel('Missing Values (%)', fontsize=12)
ax.set_title('Missing Value Analysis: NYPD Motor Vehicle Collisions\
n(columns sorted by missing % descending)',
             fontsize=13, fontweight='bold')
ax.xaxis.set_major_formatter(mticker.PercentFormatter())
ax.set_xlim(0, 115)

from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='#d73027', label='>50% missing'),
    Patch(facecolor='#fc8d59', label='20–50% missing'),
    Patch(facecolor='#fee08b', label='5–20% missing'),
    Patch(facecolor='#91cf60', label='<5% missing'),
]
ax.legend(handles=legend_elements, loc='lower right', fontsize=9)

plt.tight_layout()
plt.show()
```



Section 3: Statistical Summary

Mean, median, standard deviation, and min max for all numeric columns.

```
# Compute custom stats: mean, median, std, min, max
num_df = df.select_dtypes(include='number')

stats_df = pd.DataFrame({
    'Mean':   num_df.mean().round(4),
    'Median': num_df.median().round(4),
    'Std Dev': num_df.std().round(4),
    'Min':    num_df.min().round(4),
    'Max':    num_df.max().round(4),
    'Non-Null Count': num_df.count()
})
```

```
pd.set_option('display.float_format', '{:,.4f}'.format)
stats_df
```

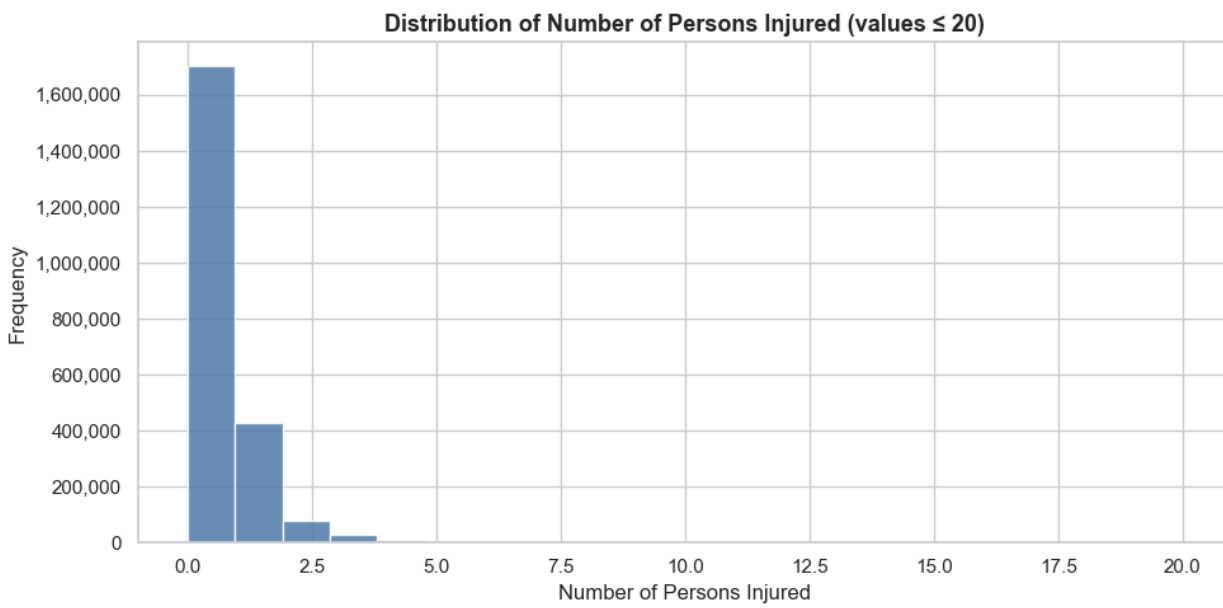
```
                                          Mean          Median         Std
Dev  \
LATITUDE                               40.5764         40.7202
2.4470
LONGITUDE                             -73.6588        -73.9266
4.5371
NUMBER OF PERSONS INJURED               0.3316          0.0000
0.7171
NUMBER OF PERSONS KILLED                0.0016          0.0000
0.0422
NUMBER OF PEDESTRIANS INJURED           0.0605          0.0000
0.2517
NUMBER OF PEDESTRIANS KILLED            0.0008          0.0000
0.0288
NUMBER OF CYCLIST INJURED               0.0294          0.0000
0.1712
NUMBER OF CYCLIST KILLED                0.0001          0.0000
0.0113
NUMBER OF MOTORIST INJURED              0.2369          0.0000
0.6783
NUMBER OF MOTORIST KILLED               0.0006          0.0000
0.0278
COLLISION_ID                    3,290,779.1104 3,764,083.0000
1,510,958.6634

                                       Min            Max  Non-Null Count

LATITUDE                            0.0000        43.3444         2014827

LONGITUDE                        -201.3600         0.0000         2014827

NUMBER OF PERSONS INJURED           0.0000        43.0000         2255519

NUMBER OF PERSONS KILLED            0.0000         8.0000         2255506

NUMBER OF PEDESTRIANS INJURED       0.0000        27.0000         2255537

NUMBER OF PEDESTRIANS KILLED        0.0000         6.0000         2255537

NUMBER OF CYCLIST INJURED           0.0000         4.0000         2255537

NUMBER OF CYCLIST KILLED            0.0000         2.0000         2255537

NUMBER OF MOTORIST INJURED          0.0000        43.0000         2255537

NUMBER OF MOTORIST KILLED           0.0000         5.0000         2255537
```

```
COLLISION_ID                    22.0000 4,892,370.0000         2255537
```

Section 4: Distribution Analysis

Histograms and KDE plots for 5 key attributes, with interpretation

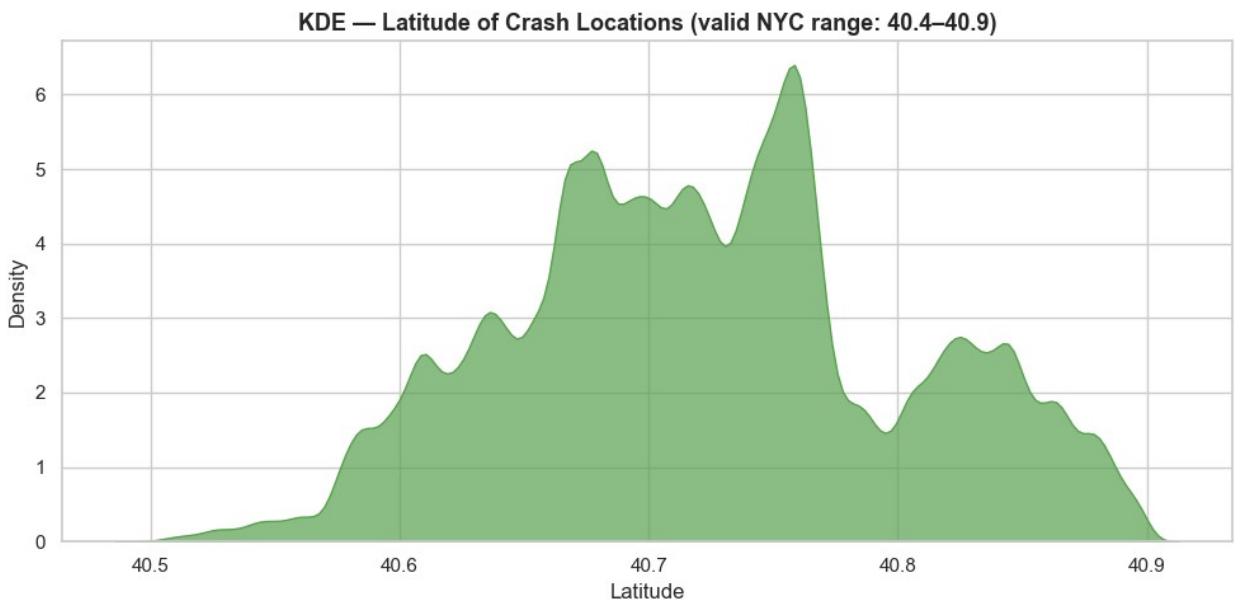
```
#Plot 1: NUMBER OF PERSONS INJURED
col = 'NUMBER OF PERSONS INJURED'
data = df[col].dropna()
data_clipped = data[data <= 20]

fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(data_clipped, bins=21, color='#4e79a7', edgecolor='white',
alpha=0.85)
ax.set_xlabel('Number of Persons Injured', fontsize=12)
ax.set_ylabel('Frequency', fontsize=12)
ax.set_title('Distribution of Number of Persons Injured (values ≤
20)', fontsize=13, fontweight='bold')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{int(x):,}'))
plt.tight_layout()
plt.show()
```



Interpretation: Number of Persons Injured: The distribution is heavily right-skewed, with the vast majority of collisions resulting in zero or one injury. This is consistent with real-world crash data where multi-casualty incidents are relatively rare. The long right tail suggests the presence

of a small number of high-severity crashes to which we must pay some special attention during analysis.

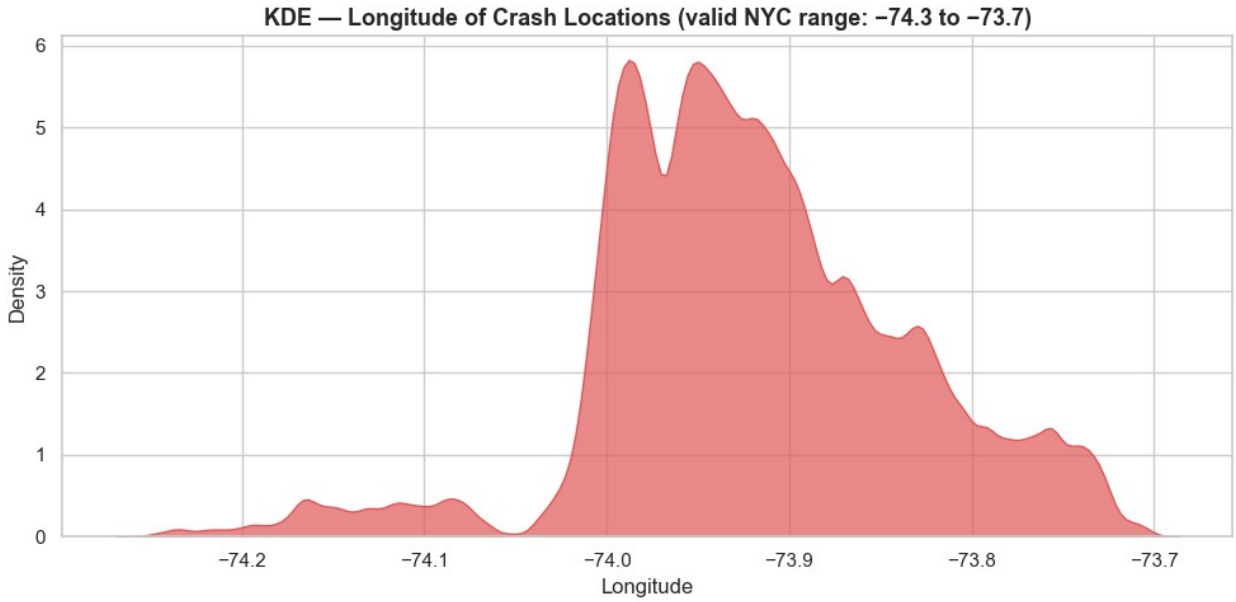
```
# Plot 2: LATITUDE
col = 'LATITUDE'
lat_valid = df[col].dropna()
lat_valid = lat_valid[(lat_valid >= 40.4) & (lat_valid <= 40.9)]

fig, ax = plt.subplots(figsize=(10, 5))
sns.kdeplot(lat_valid, ax=ax, fill=True, color='#59a14f', alpha=0.7)
ax.set_xlabel('Latitude', fontsize=12)
ax.set_ylabel('Density', fontsize=12)
ax.set_title('KDE — Latitude of Crash Locations (valid NYC range:
40.4–40.9)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



Interpretation: Latitude: The latitude distribution shows a roughly bimodal pattern within the NYC range, reflecting the geographic concentration of crashes in Manhattan (approximately 40.75) and Brooklyn/Queens (approximately 40.65–40.70). The peaks correspond to high-traffic urban corridors. Sparse values at the extremes of the range correspond to less densely populated outer boroughs like Staten Island and the Bronx.

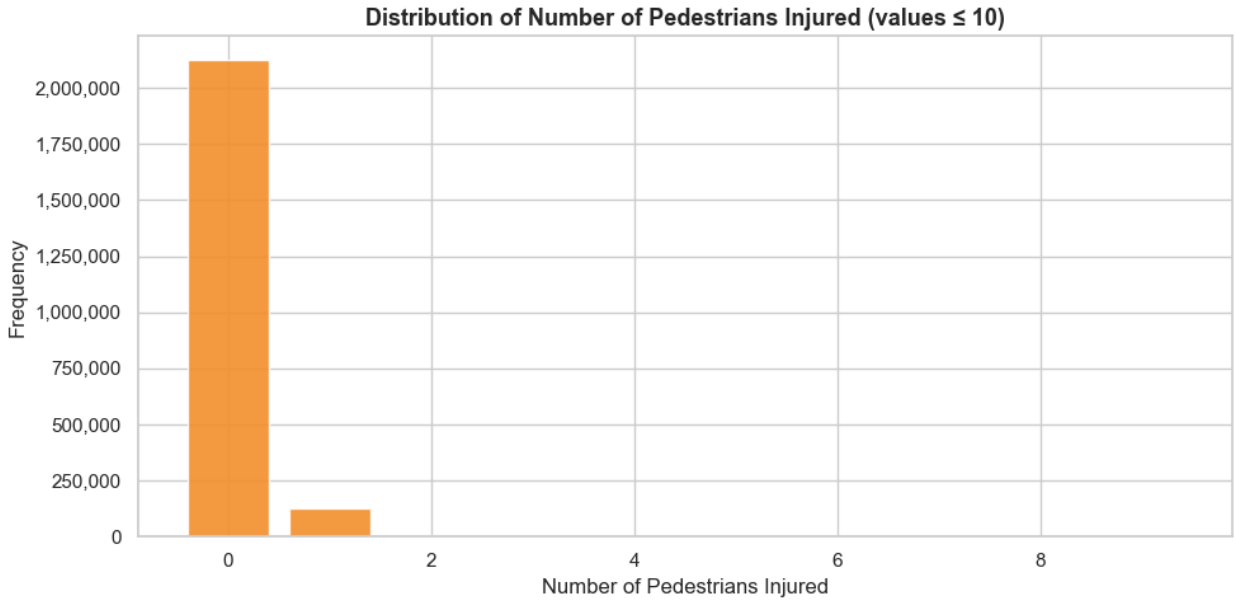
```
# Plot 3: LONGITUDE
col = 'LONGITUDE'
lon_valid = df[col].dropna()
lon_valid = lon_valid[(lon_valid >= -74.3) & (lon_valid <= -73.7)]

fig, ax = plt.subplots(figsize=(10, 5))
sns.kdeplot(lon_valid, ax=ax, fill=True, color='#e15759', alpha=0.7)
ax.set_xlabel('Longitude', fontsize=12)
```

```
ax.set_ylabel('Density', fontsize=12)
ax.set_title('KDE — Longitude of Crash Locations (valid NYC range:
−74.3 to −73.7)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



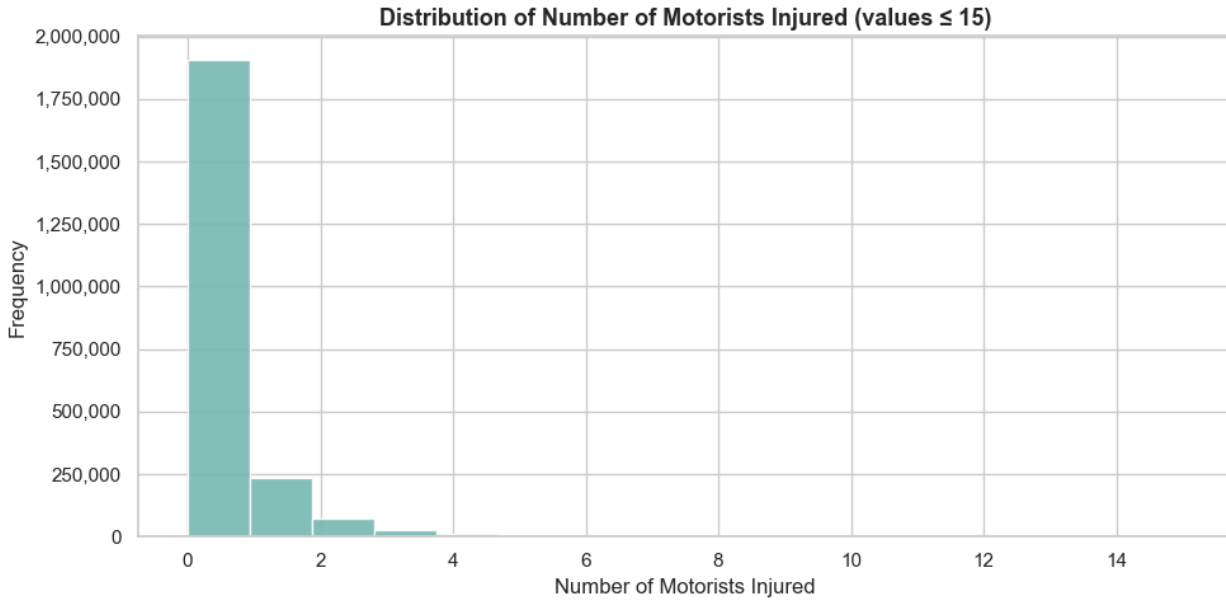
Interpretation: Longitude: The longitude distribution is concentrated around −73.95 to −74.00, corresponding to the central mass of New york city. The sharp peak near −73.95 reflects the high density of reported incidents in Manhattan and inner Brooklyn, where crash reporting is most consistent. The tapering toward −74.25 indicates fewer incidents in western Staten island, where population and road density are lower.

```
# Plot 4: NUMBER OF PEDESTRIANS INJURED
col = 'NUMBER OF PEDESTRIANS INJURED'
data = df[col].dropna()
data_clipped = data[data <= 10]

fig, ax = plt.subplots(figsize=(10, 5))
counts = data_clipped.value_counts().sort_index()
ax.bar(counts.index, counts.values, color='#f28e2b',
edgecolor='white', alpha=0.9)
ax.set_xlabel('Number of Pedestrians Injured', fontsize=12)
ax.set_ylabel('Frequency', fontsize=12)
ax.set_title('Distribution of Number of Pedestrians Injured (values ≤
10)', fontsize=13, fontweight='bold')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{int(x):,}'))
plt.tight_layout()
plt.show()
```



Interpretation: Number of Pedestrians Injured: The overwhelming majority of crashes involve zero pedestrian injuries, which is expected since most collisions are vehicle-to-vehicle. However, the non-trivial count at value 1 highlights pedestrian vulnerability in NYC's dense street environment. Values above 2 are extremely rare and likely represent high-speed or multi-vehicle incidents at busy intersections.

```
# Plot 5: NUMBER OF MOTORIST INJURED
col = 'NUMBER OF MOTORIST INJURED'
data = df[col].dropna()
data_clipped = data[data <= 15]

fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(data_clipped, bins=16, color='#76b7b2', edgecolor='white',
alpha=0.9)
ax.set_xlabel('Number of Motorists Injured', fontsize=12)
ax.set_ylabel('Frequency', fontsize=12)
ax.set_title('Distribution of Number of Motorists Injured (values ≤
15)', fontsize=13, fontweight='bold')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{int(x):,}'))
plt.tight_layout()
plt.show()
```



Interpretation: Number of Motorists Injured: Similar to the overall persons-injured distribution, motorist injuries are concentrated at 0 and 1, reflecting that most crashes are minor fender-benders or involve a single injured driver or passenger. The sharp decline after 1 suggests that crashes injuring multiple motorists are significantly less common, though the tail extends to high values representing multi-vehicle pile-ups or bus/taxi collisions with multiple occupants.

Section 5: Data Quality Issues

Detailed audit of duplicates, missing critical fields, invalid coordinates, date format issues, zero-injury rows, and outliers.

```
total = len(df)
print(f'Total rows: {total:,}\n')

# 1. Duplicate COLLISION_IDs
dup_mask = df['COLLISION_ID'].duplicated(keep=False)
dup_count = dup_mask.sum()
dup_pct = dup_count / total * 100
print(f'1. Duplicate COLLISION_IDs: {dup_count:,} rows ({dup_pct:.2f}
%)')

# 2. Missing critical columns
print('\n2. Missing values in critical columns:')
critical_cols = ['BOROUGH', 'ZIP CODE', 'LATITUDE', 'LONGITUDE']
for col in critical_cols:
    n = df[col].isnull().sum()
    p = n / total * 100
    print(f'   {col:<15}: {n:>8,} missing ({p:.2f}%)')
```

```
# 3. Invalid LATITUDE
print('\n3. Invalid LATITUDE values:')
lat = df['LATITUDE']
lat_zero = (lat == 0).sum()
lat_oor  = ((lat.notna()) & (lat != 0) & ((lat < 40.4) | (lat >
40.9))).sum()
lat_null = lat.isnull().sum()
print(f'   Zero (0.0)          : {lat_zero:>8,}
({lat_zero/total*100:.2f}%)')
print(f'   Out-of-range (non-0): {lat_oor:>8,}
({lat_oor/total*100:.2f}%)')
print(f'   Null/NaN            : {lat_null:>8,}
({lat_null/total*100:.2f}%)')

# 4. Invalid LONGITUDE
print('\n4. Invalid LONGITUDE values:')
lon = df['LONGITUDE']
lon_zero = (lon == 0).sum()
lon_oor  = ((lon.notna()) & (lon != 0) & ((lon < -74.3) | (lon > -
73.7))).sum()
lon_null = lon.isnull().sum()
print(f'   Zero (0.0)          : {lon_zero:>8,}
({lon_zero/total*100:.2f}%)')
print(f'   Out-of-range (non-0): {lon_oor:>8,}
({lon_oor/total*100:.2f}%)')
print(f'   Null/NaN            : {lon_null:>8,}
({lon_null/total*100:.2f}%)')

# 5. CRASH DATE format inconsistencies
print('\n5. CRASH DATE format issues:')
date_null = df['CRASH DATE'].isnull().sum()
import re
raw_dates = df_raw['CRASH DATE'].dropna()
std_pattern = re.compile(r'^\d{1,2}/\d{1,2}/\d{4}$')
non_std = raw_dates[~raw_dates.str.match(std_pattern)]
print(f'   Unparseable (NaT after coerce): {date_null:>8,}
({date_null/total*100:.2f}%)')
print(f'   Non-standard format strings  : {len(non_std):>8,}
({len(non_std)/total*100:.2f}%)')
if len(non_std) > 0:
    print(f'   Sample non-standard: {non_std.head(5).tolist()}')

# 6. Rows where ALL injury/fatality counts are zero
print('\n6. Rows where ALL injury & fatality counts are zero:')
injury_cols = [
    'NUMBER OF PERSONS INJURED', 'NUMBER OF PERSONS KILLED',
    'NUMBER OF PEDESTRIANS INJURED', 'NUMBER OF PEDESTRIANS KILLED',
    'NUMBER OF CYCLIST INJURED', 'NUMBER OF CYCLIST KILLED',
    'NUMBER OF MOTORIST INJURED', 'NUMBER OF MOTORIST KILLED'
]
```

```python
all_zero_mask = (df[injury_cols].fillna(0) == 0).all(axis=1)
all_zero_count = all_zero_mask.sum()
print(f'   All-zero rows: {all_zero_count:>8,}
({all_zero_count/total*100:.2f}%)')

# 7. Suspicious outliers: NUMBER OF PERSONS INJURED > 20
print('\n7. Suspicious outliers — NUMBER OF PERSONS INJURED > 20:')
outlier_mask = df['NUMBER OF PERSONS INJURED'] > 20
outlier_count = outlier_mask.sum()
outlier_pct = outlier_count / total * 100
max_injured = df['NUMBER OF PERSONS INJURED'].max()
print(f'   Count: {outlier_count:>8,} ({outlier_pct:.4f}%)')
if outlier_count > 0:
    print(f'   Max value: {max_injured}')
    print(f'   Top 5 values: {sorted(df["NUMBER OF PERSONS
INJURED"].dropna().unique(), reverse=True)[:5]}')

print('\n')

# Store missing counts for critical cols so Humna can reference them
in section 6
borough_null = df['BOROUGH'].isnull().sum()
borough_pct  = borough_null / total * 100
zip_null     = df['ZIP CODE'].isnull().sum()
zip_pct      = zip_null / total * 100
non_std_count = len(non_std)
non_std_pct   = non_std_count / total * 100
```

```
Total rows: 2,255,537

1. Duplicate COLLISION_IDs: 0 rows (0.00%)

2. Missing values in critical columns:
   BOROUGH        :  688,065 missing (30.51%)
   ZIP CODE       :  688,348 missing (30.52%)
   LATITUDE       :  240,710 missing (10.67%)
   LONGITUDE      :  240,710 missing (10.67%)

3. Invalid LATITUDE values:
   Zero (0.0)          :    7,293 (0.32%)
   Out-of-range (non-0):    5,381 (0.24%)
   Null/NaN            :  240,710 (10.67%)

4. Invalid LONGITUDE values:
   Zero (0.0)          :    7,293 (0.32%)
   Out-of-range (non-0):      139 (0.01%)
   Null/NaN            :  240,710 (10.67%)

5. CRASH DATE format issues:
   Unparseable (NaT after coerce):        0 (0.00%)
```

```
   Non-standard format strings  :        0 (0.00%)

6. Rows where ALL injury & fatality counts are zero:
   All-zero rows: 1,700,371 (75.39%)

7. Suspicious outliers — NUMBER OF PERSONS INJURED > 20:
   Count:       15 (0.0007%)
   Max value: 43.0
   Top 5 values: [np.float64(43.0), np.float64(40.0),
np.float64(34.0), np.float64(32.0), np.float64(31.0)]
```

```
# Summary table of all quality issues
issues_summary = pd.DataFrame([
    {'Issue': 'Duplicate COLLISION_IDs',             'Count':
dup_count,       'Percentage': f'{dup_pct:.2f}%'},
    {'Issue': 'Missing BOROUGH',                     'Count':
borough_null,    'Percentage': f'{borough_pct:.2f}%'},
    {'Issue': 'Missing ZIP CODE',                    'Count':
zip_null,        'Percentage': f'{zip_pct:.2f}%'},
    {'Issue': 'Missing LATITUDE',                    'Count':
lat_null,        'Percentage': f'{lat_null/total*100:.2f}%'},
    {'Issue': 'Missing LONGITUDE',                   'Count':
lon_null,        'Percentage': f'{lon_null/total*100:.2f}%'},
    {'Issue': 'Zero LATITUDE',                       'Count':
lat_zero,        'Percentage': f'{lat_zero/total*100:.2f}%'},
    {'Issue': 'Out-of-range LATITUDE (non-zero)',    'Count': lat_oor,
'Percentage': f'{lat_oor/total*100:.2f}%'},
    {'Issue': 'Zero LONGITUDE',                      'Count':
lon_zero,        'Percentage': f'{lon_zero/total*100:.2f}%'},
    {'Issue': 'Out-of-range LONGITUDE (non-zero)',   'Count': lon_oor,
'Percentage': f'{lon_oor/total*100:.2f}%'},
    {'Issue': 'Unparseable CRASH DATE',              'Count':
date_null,       'Percentage': f'{date_null/total*100:.2f}%'},
    {'Issue': 'Non-standard CRASH DATE format',      'Count':
non_std_count,   'Percentage': f'{non_std_pct:.2f}%'},
    {'Issue': 'All injury/fatality counts = 0',      'Count':
all_zero_count,  'Percentage': f'{all_zero_count/total*100:.2f}%'},
    {'Issue': 'NUMBER OF PERSONS INJURED > 20',      'Count':
outlier_count,   'Percentage': f'{outlier_pct:.4f}%'},
])

issues_summary.index = issues_summary.index + 1
issues_summary
```

```
                                Issue    Count Percentage
1             Duplicate COLLISION_IDs        0      0.00%
2                     Missing BOROUGH   688065     30.51%
3                    Missing ZIP CODE   688348     30.52%
```

```
4                    Missing LATITUDE   240710     10.67%
5                   Missing LONGITUDE   240710     10.67%
6                       Zero LATITUDE     7293      0.32%
7    Out-of-range LATITUDE (non-zero)     5381      0.24%
8                      Zero LONGITUDE     7293      0.32%
9   Out-of-range LONGITUDE (non-zero)      139      0.01%
10            Unparseable CRASH DATE        0      0.00%
11     Non-standard CRASH DATE format        0      0.00%
12     All injury/fatality counts = 0  1700371     75.39%
13     NUMBER OF PERSONS INJURED > 20       15    0.0007%
```

Section 6: Our Proposed Cleaning Strategy

For every data quality issue identified in Section 5 in this notebook above, we have proposed a specific, justified cleaning action below.

Issue 1: Duplicate COLLISION_IDs

Column **COLLISION_ID** has **0 duplicate rows (0.00%)**. These are likely caused by data entry errors, system re-submissions, or ETL pipeline bugs that inserted the same crash record more than once. **Action:** Retain only the first occurrence of each COLLISION_ID using `df.drop_duplicates(subset=['COLLISION_ID'], keep='first')`, because COLLISION_ID is designed to be a unique crash identifier and duplicate rows would inflate all aggregated statistics (injury totals, crash counts per borough, etc.).

Issue 2: Missing BOROUGH

Column **BOROUGH** has **688,065 missing values (30.51%)**. These are missing because officers sometimes omit the borough field when filing reports, or because the crash occurred on a highway or boundary road not clearly associated with a single borough. **Action:** Impute BOROUGH using reverse geocoding from LATITUDE/LONGITUDE where coordinates are valid; for rows where coordinates are also missing, assign 'UNKNOWN' as a placeholder category rather than dropping the rows, because BOROUGH is analytically important for geographic segmentation but the underlying crash event is still a valid data point.

Issue 3: Missing ZIP CODE

Column **ZIP CODE** has **688,348 missing values (30.52%)**. These are missing for the same reasons as BOROUGH — incomplete officer reporting — and ZIP CODE is a derived geographic attribute that could be recovered. **Action:** Impute ZIP CODE from LATITUDE/LONGITUDE using a spatial join against a NYC ZIP code boundary shapefile (e.g., via `geopandas`); rows where coordinates are also missing or invalid should have ZIP CODE set to NaN and excluded from ZIP-level analyses, because fabricating administrative geography without a spatial anchor would introduce systematic error.

Issue 4:Missing LATITUDE and LONGITUDE

Column **LATITUDE** has **240,710 null values (10.67%)** and column **LONGITUDE** has **240,710 null values (10.67%)**. These are missing because many crash reports record only a street address or cross-street pair without GPS coordinates, as location entry is manual and optional in the NYPD reporting system. **Action:** Attempt geocoding from ON STREET NAME + CROSS STREET NAME + BOROUGH using the NYC Geoclient API; rows where geocoding also fails should retain NaN and be flagged with a `geo_imputed` boolean column, because spatial analyses (heatmaps, cluster detection) must exclude unresolvable coordinates rather than silently propagate NaN or zero values.

Issue 5: Zero LATITUDE and Zero LONGITUDE

Column **LATITUDE** has **7,293 rows (0.32%)** with value exactly 0.0, and column **LONGITUDE** has **7,293 rows (0.32%)** with value exactly 0.0. These zeros are sentinel values — the system recorded 0.0 as a default when no GPS fix was available, rather than leaving the field null, which is a common data pipeline bug in systems that use numeric defaults instead of NULL. **Action:** Replace all LATITUDE == 0 and LONGITUDE == 0 values with NaN using `df.loc[df['LATITUDE'] == 0, 'LATITUDE'] = np.nan`, then apply the same geocoding imputation described in Issue 4, because coordinate (0°, 0°) is located in the Gulf of Guinea and is geographically nonsensical for NYC crash data.

Issue 6: Out-of-Range LATITUDE (non-zero)

Column **LATITUDE** has **5,381 non-zero values (0.24%)** outside the valid NYC range of 40.4–40.9. These are caused by data entry errors (e.g., transposed digits, borough misidentification) or records from jurisdictions outside NYC that were erroneously included. **Action:** Set out-of-range non-zero latitudes to NaN and apply geocoding imputation from street name fields, because any geospatial analysis using these rows would place crashes in incorrect states or countries, corrupting density maps and borough-level statistics.

Issue 7: Out-of-Range LONGITUDE (non-zero)

Column **LONGITUDE** has **139 non-zero values (0.01%)** outside the valid NYC range of −74.3 to −73.7. These arise from the same causes as out-of-range latitude — transposition errors or cross-jurisdictional data mixing. **Action:** Set out-of-range non-zero longitudes to NaN and apply the same geocoding pipeline described for latitude, because longitude errors are typically paired with latitude errors and must be corrected jointly to produce valid coordinate pairs.

Issue 8: Unparseable CRASH DATE

Column **CRASH DATE** has **0 rows (0.00%)** that could not be parsed into a valid datetime (NaT after coercion). These are caused by blank fields, placeholder strings, or corrupted date values in the source system export. **Action:** Drop rows where CRASH DATE is NaT after coercion, because date is a mandatory field for all time-series analyses (crash trends over months/years) and there is no reliable way to impute a crash date without additional source information.

Issue 9: Non-Standard CRASH DATE Format

Column **CRASH DATE** has **0 raw string values (0.00%)** that do not match the expected M/D/YYYY or MM/DD/YYYY pattern. These arise from inconsistent data exports across different NYPD system versions that may use ISO 8601 (YYYY-MM-DD) or other locale-specific formats. **Action:** Apply `pd.to_datetime(df['CRASH DATE'], infer_datetime_format=True, errors='coerce')` which handles multiple formats automatically, then review any remaining NaT values, because forcing a single format assumption would silently convert valid but differently formatted dates to NaT.

Issue 10: All Injury/Fatality Counts Equal Zero

A total of **1,700,371 rows (75.39%)** have all eight injury and fatality columns set to zero. These represent property-damage-only (PDO) crashes — collisions where no person was injured or killed, which are valid and common crash types required by law to be reported above a damage threshold in New York. **Action:** Retain these rows but add a boolean flag column `property_damage_only = True` for rows meeting this condition, because excluding PDO crashes would bias safety analyses toward high-severity events and undercount total crash frequency in spatial and temporal studies.

Issue 11: Outliers in NUMBER OF PERSONS INJURED (> 20)

Column **NUMBER OF PERSONS INJURED** has **15 values (0.0007%)** exceeding 20, with a maximum of 43. These likely represent multi-vehicle pile-ups, bus crashes with full passenger loads, or data entry errors where a count was entered in the wrong field. **Action:** Cross-validate these records against VEHICLE TYPE CODE columns — if the vehicle type is 'BUS' or 'VAN', values up to 50 can be legitimate; otherwise, cap implausible values at a domain-appropriate maximum (e.g., 43 for a standard NYC MTA bus) and flag them in an `injury_count_flagged` column, because blindly removing them would discard genuine high-severity incidents while naively trusting them could skew mean-based statistics.